

제5장 — 토큰나이저 모델을 써서 학습하여 토큰화 시킨 출력 결과

1. 토큰화(Tokenization)의 필요성

자연어는 기계가 이해하기 어려운 형태의 비정형 데이터다.

예를 들어 “자연어 처리는 정말 재미있다.”라는 문장은 사람은 단어 단위로 구분해 이해하지만,

컴퓨터는 문자 하나하나를 숫자로 처리해야 한다.

따라서 문장을 **의미 단위로 쪼개는 과정**, 즉 **토큰화(tokenization)** 가 NLP의 첫 단계가 된다.

이 장에서는 단순한 단어 분리 수준을 넘어서,

토큰나이저 모델(tokenizer model) 자체를 학습시켜

언어의 통계적 규칙을 자동으로 학습하는 방법을 다룬다.

2. 토큰화의 여러 가지 접근 방법

(1) 단어 기반 토큰화 (Word-level Tokenization)

- 문장을 공백 기준으로 단어 단위로 나누는 가장 기본적인 방식
- 장점: 직관적, 구현 간단
- 단점: 희귀 단어나 오타자에 취약, 사전에 없는 단어(OOV, Out-Of-Vocabulary) 처리 불가

(2) 문자 기반 토큰화 (Character-level Tokenization)

- 모든 문자를 개별 토큰으로 취급
- 장점: OOV 문제 해결
- 단점: 문맥 정보 손실, 시퀀스 길이 증가

(3) 하위 단어(Subword) 기반 토큰화

- 단어와 문자 사이의 절충형 접근법
- 희귀 단어를 더 작은 단위로 쪼개고,
자주 등장하는 조합은 하나의 토큰으로 유지
- 대표 알고리즘: **BPE(Byte Pair Encoding)**, **WordPiece**, **Unigram**

3. 주요 토큰화 알고리즘

(1) BPE (Byte Pair Encoding)

- 데이터 압축 기법에서 유래한 토큰화 방식
- 가장 빈번하게 등장하는 문자 쌍을 병합하여 새로운 토큰을 생성
- 'lo', 've' → 'love' 식으로 점진적 병합
- WordPiece, SentencePiece 등 현대 모델의 근간이 됨

(2) WordPiece (BERT에서 사용)

- BPE와 유사하지만, 병합 기준이 빈도 대신 **언어모델의 우도(likelihood)**
- 즉, 병합 시 언어모델의 확률이 최대화되도록 병합

(3) Unigram (SentencePiece에서 사용)

- 미리 정해진 어휘 집합(candidate vocabulary)에서 주어진 문장에 대해 가장 높은 확률을 부여하는 분절(segment)을 선택
- 단어 쪼개기보다 **삭제형 접근법**을 사용

4. 토큰나이저 모델 학습 방법

책에서는 HuggingFace의 **Tokenizers** 라이브러리를 이용해 사용자 데이터셋으로 직접 **BPE 모델을 학습시키는 과정**을 설명한다.

핵심 코드 흐름 예시:

```
from tokenizers import Tokenizer, models, trainers, pre_tokenizers
```

```
# 1) 빈 토큰나이저 초기화
```

```
tokenizer = Tokenizer(models.BPE())
```

```
# 2) 학습용 pre-tokenizer 정의 (공백 단위 등)
```

```
tokenizer.pre_tokenizer = pre_tokenizers.Whitespace()
```

```
# 3) Trainer 정의 (어휘 크기 설정)
```

```
trainer = trainers.BpeTrainer(vocab_size=30000, special_tokens=["<unk>", "<pad>", "<bos>", "<eos>"])
```

4) 학습 실행

```
tokenizer.train(["dataset.txt"], trainer)
```

5) 학습된 토큰나이저 저장 및 테스트

```
tokenizer.save("bpe_tokenizer.json")
```

```
print(tokenizer.encode("자연어 처리는 흥미롭다").tokens)
```

이 과정을 통해 **말뭉치 기반으로 자동 어휘 사전을 구성**하고,
데이터에 최적화된 토큰 단위로 문장을 분해할 수 있다.

5. 토큰화 결과 해석

학습된 토큰나이저로 문장을 입력하면 다음과 같은 결과가 나온다.

입력: "나는 자연어 처리를 공부합니다."

출력 토큰: ["나", "는", "자연어", "처리", "##를", "공부", "##합", "##니다", "."]

- 접두사 ##는 "이전 토큰에 이어진 하위 단어(subword)"임을 나타냄.
- 모델은 이렇게 쪼개진 단위를 벡터로 변환해 학습한다.

6. 핵심 포인트 요약

항목	설명
핵심 목표	토큰나이저를 직접 학습시켜 데이터에 맞는 토큰 체계 구축
주요 알고리즘	BPE, WordPiece, Unigram
도구	HuggingFace tokenizers 라이브러리
결과	효율적인 어휘 집합, 낮은 OOV 비율, 고성능 언어모델 입력용 데이터 생성

제6장 — CBOW 모델과 Skip-gram 모델 시각화

1. 단어 임베딩의 개념

5장에서 텍스트를 토큰 단위로 분리했다면,

6장에서는 이를 **벡터(숫자)** 형태로 변환하는 **임베딩(embedding)** 과정을 다룬다.

단어를 단순히 정수 인덱스로 표현하는 것은 단어 간 의미 관계를 반영하지 못한다.

따라서 **의미적으로 비슷한 단어는 비슷한 벡터로 표현되도록 학습**하는 것이 임베딩의 핵심이다.

2. Word2Vec의 핵심 아이디어

Word2Vec은 단어를 벡터로 학습시키는 대표적인 신경망 언어모델이다.

그 중 두 가지 구조가 핵심이다:

(1) CBOW (Continuous Bag of Words)

- 주변 단어(Context)로부터 중심 단어(Target)을 예측
- 입력: 주변 단어들
- 출력: 중심 단어
- 빠른 학습 속도, 데이터가 많을수록 유리

(2) Skip-gram

- 중심 단어로부터 주변 단어를 예측
- 입력: 중심 단어
- 출력: 주변 단어
- 희귀 단어 학습에 유리

3. Word2Vec 모델 구조

간단히 Skip-gram 구조를 예로 들면:

입력: "I love NLP"

중심 단어: "love"

예측 대상: ["I", "NLP"]

학습 목표는

$P(\text{context_word} \mid \text{target_word})$ 를 최대화하는 것이다.

네트워크는 얇은 2-layer 구조로 되어 있으며,

입력층 → 임베딩층 → 출력층 순서로 구성된다.

4. 학습 원리와 수학적 표현

Word2Vec의 손실 함수는 다음과 같다:

$$L = - \sum_{(w,c) \in D} \log P(c | w)$$

여기서 $P(c | w)$ 는 softmax 확률이며,
모든 단어에 대한 확률을 계산하기엔 너무 비싸므로,
Negative Sampling을 도입하여 연산량을 줄인다.

5. 임베딩 벡터 시각화

학습된 단어 벡터는 고차원(예: 100~300차원)이므로
PCA나 t-SNE로 2D 평면에 투영하여 시각화한다.

예시 결과:

- 'king'과 'queen', 'man'과 'woman' 벡터가 평행한 방향
- '서울'과 '한국', '파리'와 '프랑스'가 가까운 거리로 배치됨

이러한 결과를 통해 Word2Vec이 **의미적 유사성을 수치적으로 표현**함을 알 수 있다.

6. 실습 — Gensim으로 Word2Vec 학습

책에서는 **Gensim** 라이브러리를 활용해 간단히 모델을 학습하고 시각화하는 방법을 보여준다.

```
from gensim.models import Word2Vec
```

```
sentences = [["나는", "자연어", "처리를", "공부한다"], ["자연어", "처리", "흥미롭다"]]
```

```
model = Word2Vec(sentences, vector_size=100, window=3, min_count=1, sg=1) # sg=1  
# 은 Skip-gram
```

```
model.wv.most_similar("자연어")
```

7. RNN, LSTM, CNN, Transformer와의 비교

이 장에서는 과거의 **Word2Vec 기반 임베딩** 모델에서 **문맥(context)** 을 동적으로 처리하는 **RNN/LSTM/Transformer** 계열로 언어모델이 진화하는 과정을 간략히 소개한다.

- **RNN**: 순차적 구조로 문맥 반영 가능하지만, 장기 의존성 문제 존재
- **LSTM/GRU**: RNN 개선 버전으로 장기 의존성 해결
- **CNN**: 지역 단어 패턴을 포착 (문장 분류 등에 유용)
- **Transformer**: Self-Attention으로 문맥을 병렬적으로 처리, 최신 모델의 주류

또한, **Optimizer의 발전**(SGD → Momentum → Adam)과 **Adam의 적응적 학습률(adaptive learning rate)** 개념도 비교하며 현대 신경망 학습의 핵심 변화를 설명한다.